

Procedures Implemented for US17 and US18

US17

```
private static ArrayList<Edge> readEdgesFromFileUS17(String matrixFilePath, String pointsFilePath) throws FileNotFoundException {
    ArrayList<Edge> edges = new ArrayList<>();
    ArrayList<Point> points = new ArrayList<>();
    Scanner scPoints = new Scanner(new File(pointsFilePath));
    if (scPoints.hasNextLine()) {
        String line = scPoints.nextLine();
        String[] pointNames = line.split(" ");
        for (String pointName : pointNames) {
            points.add(new Point(pointName));
        }
    }
    scPoints.close();
    Scanner scMatrix = new Scanner(new File(matrixFilePath));
    int row = 0;
    while (scMatrix.hasNextLine()) {
        String line = scMatrix.nextLine();
        line = line.substring(1);
        String[] weights = line.split(" ");
        for (int col = 0; col < weights.length; col++) {
            int weight = Integer.parseInt(weights[col]);
            if (weight > 0) {
                edges.add(new Edge(points.get(row), points.get(col), weight));
            }
        }
        row++;
    }
    scMatrix.close();

    return edges;
}
```

Figure 1 – Method to read the edges from the files

```
public static ArrayList<Edge> dijkstraAlgorithmUS17(ArrayList<Edge> edges, ArrayList<Point> vertices, Point startingPoint, Point endPoint) {
    ArrayList<Edge> resultPath = new ArrayList<>();

    int numVertices = vertices.size();

    //lista dos custos
    ArrayList<Integer> costsOfVertices = new ArrayList<>(Collections.nCopies(numVertices, Integer.MAX_VALUE));
    //lista para saber se o vértice já foi usado
    ArrayList<Boolean> verticesUsed = new ArrayList<>(Collections.nCopies(numVertices, false));
    //lista do caminho usado
    ArrayList<Point> pathVertices = new ArrayList<>(Collections.nCopies(numVertices, null));

    int indexOfStartingPoint = vertices.indexOf(startingPoint);
    int indexOfEndPoint = vertices.indexOf(endPoint);

    costsOfVertices.set(indexOfStartingPoint, 0);

    for (int i = 0; i < numVertices - 1; i++) {
        //dos vértices com os quais o vértice escolhido tem ligação, o index do que tem menor custo
        int indexOfMinimum = indexOfMin(costsOfVertices, verticesUsed, numVertices);
        //o algoritmo pára quando o de menor custo for o endPoint ou quando todos os vértices já foram usados
        if (indexOfMinimum == -1 || indexOfMinimum == indexOfEndPoint) {
            break;
        }

        Point chosenVertex = vertices.get(indexOfMinimum);
        verticesUsed.set(indexOfMinimum, true);
    }
}
```

Figure 2 – Method to determine the shortest path with the Dijkstra Algorithm

```

//atualizar os custos dos caminhos (no caso de serem menores)
for (Edge edge : edges) {
    if (edge.getP1().equals(chosenVertice)) {
        int indexOfNext = searchForIndexPoint(vertices,edge.getP2());
        if (!verticesUsed.get(indexOfNext)){
            int cost=costsOfVertices.get(indexOfMinimum)+edge.getPrice();
            if (cost<costsOfVertices.get(indexOfNext)) {
                costsOfVertices.set(indexOfNext,cost);
                pathVertices.set(indexOfNext,chosenVertice);
            }
        }
    }
}

// construir o caminho resultante
Point currentPoint=endPoint;
while (currentPoint != null && !currentPoint.equals(startingPoint)) {
    int indexOfCurrent=searchForIndexPoint(vertices,currentPoint);
    Point beforePoint=pathVertices.get(indexOfCurrent);
    if (beforePoint == null) {
        break;
    }
    int cost = costsOfVertices.get(vertices.indexOf(currentPoint))-costsOfVertices.get(vertices.indexOf(beforePoint));
    resultPath.add(new Edge(beforePoint, currentPoint, cost));
    currentPoint=beforePoint;
}

return resultPath;
}

```

Figure 3 – Method to determine the shortest path with the Dijkstra Algorithm

```

private static int indexOfMin(ArrayList<Integer> costsOfVertices, ArrayList<Boolean> verticesVisited, int numVertices) {
    int min = Integer.MAX_VALUE;
    int indexOfMinimum = -1;
    for (int j = 0; j < numVertices; j++) {
        if (!verticesVisited.get(j) && costsOfVertices.get(j) < min) {
            min = costsOfVertices.get(j);
            indexOfMinimum = j;
        }
    }
    return indexOfMinimum;
}

```

Figure 4 – Method to determine the index of the vertex with the lowest cost

```

public static int searchForIndexPoint(ArrayList<Point> points, Point point) {
    for (int i = 0; i < points.size(); i++) {
        if (points.get(i).equals(point)) {
            return i;
        }
    }
    return -1;
}

```

Figure 5 – Method to determine the index of the point given by argument

```

public static void createResultFile(ArrayList<Edge> edges) throws IOException {
    FileWriter result = new FileWriter("result.csv");
    int price = 0;
    StringBuilder path = new StringBuilder();
    for (int i = 0; i < edges.size() - 1; i++) {
        Edge e = edges.get(i);
        if (i == 0) {
            path.append(e.getP1()).append(";");
        }
        path.append(e.getP2()).append(";");
        price += edges.get(i).getPrice();
    }
    Edge lastEdge = edges.get(edges.size() - 1);
    path.append(lastEdge.getP2());
    String line1 = String.format("%s,%d\n", path.toString(), lastEdge.getPrice());
    result.write(line1);
    result.close();
}

```

Figure 6 - Method to output the csv file containing the result path

US18

To solve the problem given in the User Story, the algorithm implemented in US17 was used along with two additional methods to determine the path with the lowest cost, thereby identifying the closest Assembly Point.

```
private static ArrayList<Edge> comparingResults(ArrayList<ArrayList<Edge>> results) {
    ArrayList<Edge> path=new ArrayList<>();
    int cost=0;
    ArrayList<Integer> costs=new ArrayList<>();
    for(ArrayList<Edge> array : results){
        for(Edge e : array){
            cost+=e.getPrice();
        }
        costs.add(cost);
        cost=0;
    }

    int minCost=costs.get(0);
    int posMinCost=0;
    for(Integer i : costs){
        if(i<minCost){
            minCost=i;
            posMinCost=searchForIndexCost(costs,minCost);
        }
    }

    path=results.get(posMinCost);
    return path;
}
```

Figure 7 - Method to determine the path with the least cost

```
public static int searchForIndexCost(ArrayList<Integer> costs, int cost) {
    for (int i = 0; i < costs.size(); i++) {
        if (costs.get(i) == cost) {
            return i;
        }
    }
    return -1;
}
```

Figure 8 - Method to search for the index of the lowest cost